

Runstop

나만의 러닝 코스



3조 박건희 | 윤재빈 | 이승연 | 최한빈 | 허완

팀 소개



허완

프로젝트 구조 관리 및 테스트



박건희

알고리즘 및 AI 개발



윤재빈

데이터 전처리 및 알고리즘



이승연

사용자 흐름 설계 및 프론트 UI/UX



최한빈

프론트UI/UX 및 병합

목 차

01 프로젝트 소개

02 서비스와 시스템 설계

03 코스 생성 알고리즘

04 모델별 성능 비교

05 시연& 트러블 슈팅

01. 프로젝트 개요

개요 - 배경

러닝 열풍, 이번엔 아기와 함께...부산 북항서 '베이비런'

김희윤 기자

입력 2026.08.31 16:14

 선호 매체 추가

“이젠 산으로 가요”...러닝 열풍에 ‘트레일러닝’ 뜬다

러닝 인구 1000만...자연 뛰는 트레일러닝 대세
대회는 10분 만에 조기 마감되기도
아웃도어 브랜드, 트레일러닝 상품 선보여

이채원 기자 | 입력 : 2026.04.24 17:00

러닝화 빌려주고 탈의실도... '달리기 천국' 서울

김윤주 기자 · 2025. 9. 9. 00:34

유통식품업계,러닝열풍타고체험마케팅확대

 차미경 |  승인 2026.09.11 09:50 |  댓글 0

롯데 1만5000명 규모...hy·삼양도 러닝 확대
제품 체험·SNS 인증 결합...브랜드 접점 확대

기존 서비스의 한계

01

러닝에 필요한 정보를 따로 확인해야 함

거리·경사도는 기존 러닝 서비스에서 확인할 수 있지만,
화장실·편의점·야간 인프라는 별도로 찾아야 합니다

02

여러 조건을 만족하는 코스를 비교하기 어려움

목표 거리와 경사, 시설, 야간 안전 조건을 함께 반영해
어떤 코스가 더 적합한지 판단하기 어렵습니다.



사용자는 코스를 찾기 위해 여러 정보를 직접 조합하고 비교해야 합니다.

Runstop의 해결 방식

01

조건 입력

출발지와 목표 거리를 입력하고
목적지 여부, 시설, 경사도, 야간 조건을
설정합니다

02

후보 경로 생성 · 평가

보행 네트워크에서 경로 후보를 생성하고
거리, 경사, 시설, 야간 조건을 점수화합니다

03

맞춤 추천

조건 충족도를 기준으로 코스를 비교해
거리, 예상 시간, 추천 이유와
함께 제공합니다

02. 서비스와 시스템 설계

서비스 아키텍처 & 기술 스택

개발 언어 / 공통 라이브러리



TypeScript



Zod

클라이언트

React
React Native

백엔드



Node.js



FastAPI

데이터베이스



PostgreSQL

인프라 / 배포



Route53



EC2

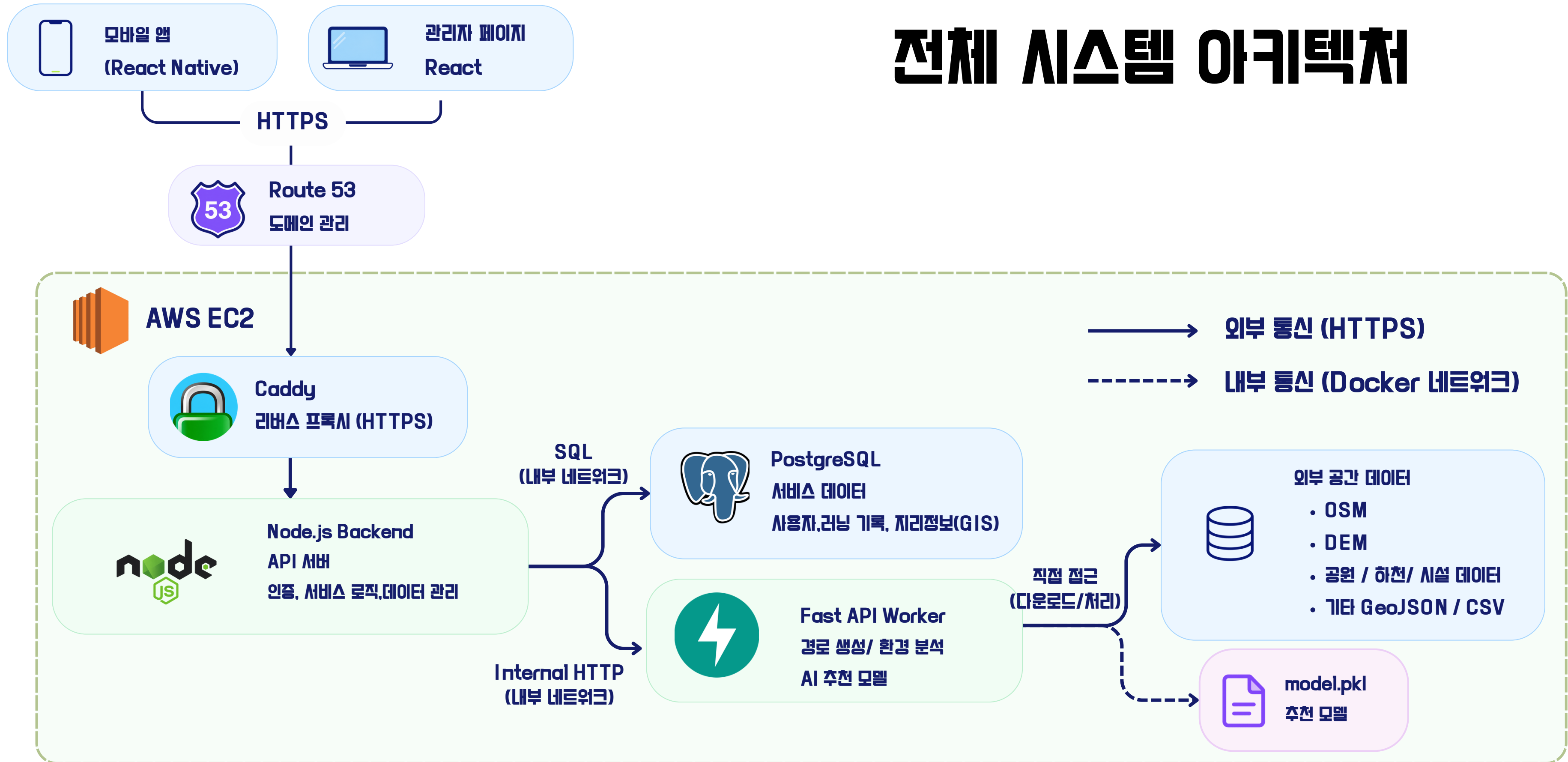


Docker



Caddy

전체 시스템 아키텍처



핵심 ERD

사용자

users		사용자 기본 정보
 idx	integer	사용자 ID
 login_id	varchar(50)	로그인 ID
 nickname	varchar(30)	닉네임
 total_exp	integer	총 경험치
 role	enum	사용자 권한
 status	enum	계정 상태
 created_at	timestampz	생성 시각

user_profiles		사용자 프로필 정보
 idx	integer	프로필 ID
 users_idx	integer	사용자 ID
 weight	numeric(5,2)	몸무게
 height	numeric(5,2)	키
 running_settings	jsonb	러닝 설정

경로 요청

route_requests		사용자 경로 요청 정보
 idx	integer	요청 ID
 users_idx	integer	사용자 ID
 prompt	text	자연어 요청
 route_type	varchar(20)	경로 유형
 element_conditions	jsonb	세부 조건
 selected_recommendations_idx	integer	선택 추천 ID
 created_at	timestampz	생성 시각

route_request_points		경로 요청 좌표 정보
 idx	integer	요청 좌표 ID
 route_requests_idx	integer	요청 ID
 sequence	integer	순서
 point_type	enum	지점 유형
 point	geometry	좌표 정보

경로 추천

route_recommendations		추천 경로 후보 정보
 idx	integer	추천 ID
 route_requests_idx	integer	요청 ID
 name	varchar(100)	경로 이름
 score	numeric(6,3)	추천 점수
 total_distance	integer	총 거리
 total_ascent	numeric(8,2)	총 상승고도
 route	geometry	경로 데이터
 created_at	timestampz	생성 시각

route_points		추천 경로 주요 지점
 idx	integer	지점 ID
 route_recommendations_idx	integer	추천 경로 ID
 sequence	integer	순서
 title	varchar(100)	지점명
 point_type	enum	지점 유형
 elevation	numeric(8,2)	고도
 slope	numeric(7,3)	경사
 point	geometry	좌표 정보

실시간 러닝

running_sessions		러닝 세션 기록
 idx	integer	세션 ID
 users_idx	integer	사용자 ID
 route_recommendations_idx	integer	추천 경로 ID
 status	enum	진행 상태
 started_at	timestampz	시작 시각
 finished_at	timestampz	종료 시각
 distance	integer	실제 거리
 average_pace	integer	평균 페이스

running_trackpoints		실시간 GPS 기록
 idx	integer	트랙포인트 ID
 client_trackpoint_id	uuid	클라이언트 ID
 running_sessions_idx	integer	세션 ID
 point	geometry	좌표 정보
 recorded_at	timestampz	기록 시각
 accuracy	numeric(8,2)	정확도

사용자 시나리오

퇴근하고 5km 정도 달리고 싶은데
밤에도 안전한 길이면 좋겠어



사용자 요청이 처리되는 과정



요청 처리

1. 사용자가 출발지, 거리, 경사, 야간 조건을 선택
2. 서버가 요청값을 검증하고 경로 생성 요청을 전달



경로 처리

1. 조건에 맞는 후보 경로 생성
2. 거리 · 경사 · 시설 · 안전 조건 점수 계산
3. 추천 코스와 추천 이유를 앱에 반환

03. 코스 생성 알고리즘

전체 알고리즘 파이프라인

1

후보 경로 생성

MultiDiGraph와
시설 anchor 사용

2

후보 필터링

거리·겹침 제거

3

Feature 추출

경사·시설 / 자연·노면
적용 여부와 점수

4

휴리스틱 점수화

feature_score
→ condition_score

5

AI 후보 랭킹

선택된 모델을 통한
점수기반 정렬

6

Top-k 추천

추천 후보 3개 반환

사용자의 weights·requirements가 후보 생성부터 점수화까지 그대로 전달됩니다.

코스 생성에 사용되는 Feature 4가지

경사 (Slope)



데이터

서울 등고선 데이터

방식

경로를 30m 간격으로 샘플링 → 구간 경사 계산
→ 평균·최대·표준편차, 상승·하강도 산출

시설 (Facilities)



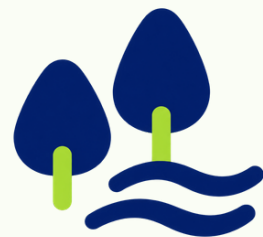
데이터

8종 시설물 위경도 데이터

방식

최근접 거리 + km당 개수(밀도) 산출

자연(Nature)



데이터

공원·하천 GeoJSON

방식

경로 인접 구간 비율
(park_ratio, water_ratio) 계산

노면 (Surface)



데이터

OSM 도로 데이터

방식

보행 가능 비율, 계단 개수, 신호·횡단보도
밀도 계산

경사,시설,자연은 좌표(coords) 기반, 노면만 그래프의 도로 속성 (MultiDiGraph)을 직접 읽어 계산합니다

개인화된 경로 탐색

$$\text{Cost(도로)} = \text{실제 길이} \times \text{개인화 배율}$$

개인화 배율

$$1 + \sum(\text{가중치} \times \text{패널티})$$

가중치	패널티
경사	경사 패널티
안전	조명·CCTV 패널티
자연	공원·하천 패널티
노면	계단·신호 패널티

1



개인화

사용자 입력(0-5)를 0~1로 정규화해
각 요소의 가중치로 반영

2



다양성 확보

이미 지나간 도로에는 추가 패널티를 부여해
추천후보끼리 겹치지 않도록 유도

3



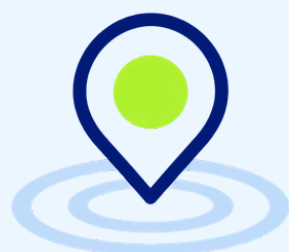
제약 조건

계단 회피, 최대 경사 등은 탐색단계에서부터
hard constraint로 반영



같은 출발점, 같은 목표 거리라도 취향에 따라 그래프는 다른 경로를 찾습니다

후보 필터링



10%

거리 오차 허용



0.35

최대 겹침 허용



0.6

중복 판정 기준



어떻게 걸러내는가?

- 두 후보 경로의 edge 집합 교집합 비율이 중복 판정 기준 (0.6)을 넘으면 같은 코스로 간주해 뒤 후보 제거
- out_and_back(왕복)은 구조상 같은 길을 그대로 되돌아오므로 겹침 하드컷에서 예외 처리

비용이 큰 연산(DEM-OSM 조회)을 모든 후보에 돌리기 전에 저렴한 기하학적 필터로 먼저 추립니다

후보 경로 생성

목적지 설정

이동형 코스

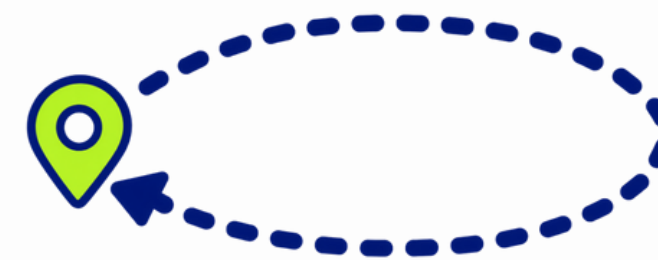


출발지에서 지정한 목적지까지 이동

ONE_WAY

목적지 미설정

순환형 코스



다른 길을 통해 출발지로 다시 돌아옴

LOOP

목적지 유무와 복귀 방식에 따라 두 가지 경로 유형을 생성합니다

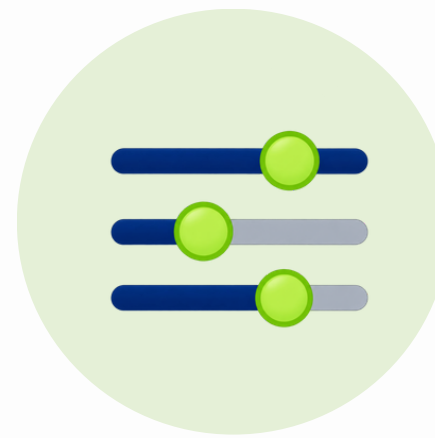
후보 경로 평가 및 추천

생성된 후보 경로를 동일한 기준으로 평가하고, 사용자 선호에 가장 맞는 상위 3개 코스를 추천합니다



항목별 점수화

- 거리 적합성 · 경사 · 노면 상태
- 시설 접근성 · 자연환경 · 야간 안전성
- 보행 흐름 · 경로 중복도
- 각 평가 항목을 0~100점 기준으로 점수화



사용자별 선호 반영

- 필수: 충족하지 않으면 후보 제외
 - 우선 : 높은 가중치
 - 선호: 기본 / 낮은 가중치
 - 미반영 : 추천 점수에서 제외
- 가중치를 반영해 최종 condition_score 산출



추천 결과 제공

- 필수 조건을 만족한 후보만 비교
- condition_score 기준으로 후보 순위 결정
- 상위 3개 코스의 거리·예상 시간·조건별 점수 제공

점수 기준 튜닝 및 검증

상수	값	근거
DIST_ERR_ZERO_PCT	10% (유지)	하드컷과 값 일치 필요
GAIN_PER_KM_ZERO	30m/km (유지)	2차 실측 재확인
TOILET_FULL/ STORE_FULL	2/3 → 3/7	raw count 기준 교정
WATER_RATIO_FULL	0.15 → 0.05	관측 최댓값보다 높아 도달 불가
WALKABLE_FULL	0.60 → 0.90	0.79보다 낮아 상시 만점이 존재
SIGNAL_PER_KM_ZERO	4.0 → 1.0	관측 최댓값보다 높아 상시 고득점

Valhalla Pedestrian Costing의 보행 경로 회피 원칙 참고,서울 보행 네트워크에서 생성한 후보 경로 507개 분석

04. 모델별 성능 비교

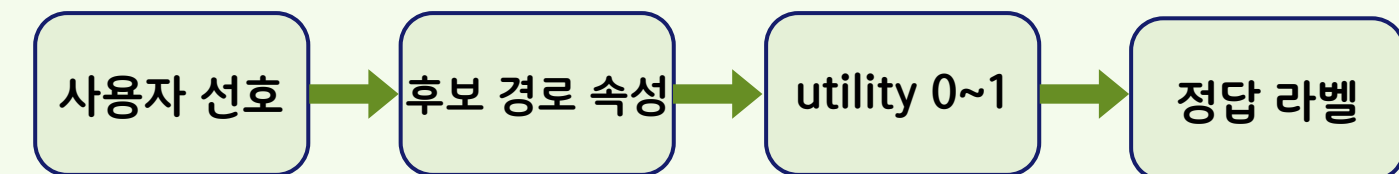
학습 데이터 구축과 합성 정답 생성

학습 데이터셋 구성

항목	규모
합성 사용자	1,000명
원본 요청	5,000명
후보 생성 성공 요청	3,292건
후보 경로 데이터	31,693개
요청당 후보 수	6~10개, 평균 9.63개

- 공간 자료: 서울 보행 네트워크, 10m DEM, 시설 데이터, 공원과 하천 데이터
- 후보 처리: 동일 경로 중복 제거, 최대 개수 초과 시 seed 기반 샘플링
- 후보 부족 및 생성 오류 1,708건 제외, 유효 요청률 65.84%

합성 utility와 정답 라벨



정답 정보	의미
Utility	후보 경로의 합성 적합도 점수
Ground-truth rank	Utility가 높은 순서의 정답 순위
Relevance	Utility를 0~5 단계로 변환한 라벨

공통 실험 설정과 평가 기준

공통 입력 피쳐

입력 영역	개수	주요 정보
사용자 프로필	11	거리·경사·편의시설·안전 등에 대한 선호
요청 정보	16	목표 거리, 경유지 수, 경로 유형, 요청별 가중치
필수 조건	5	화장실·편의점·공원·계단·최대 경사 조건
후보 경로 정보	43	경사, 거리, 시설, 자연환경, 도로·보행 특성

사용자 단위 데이터 분할

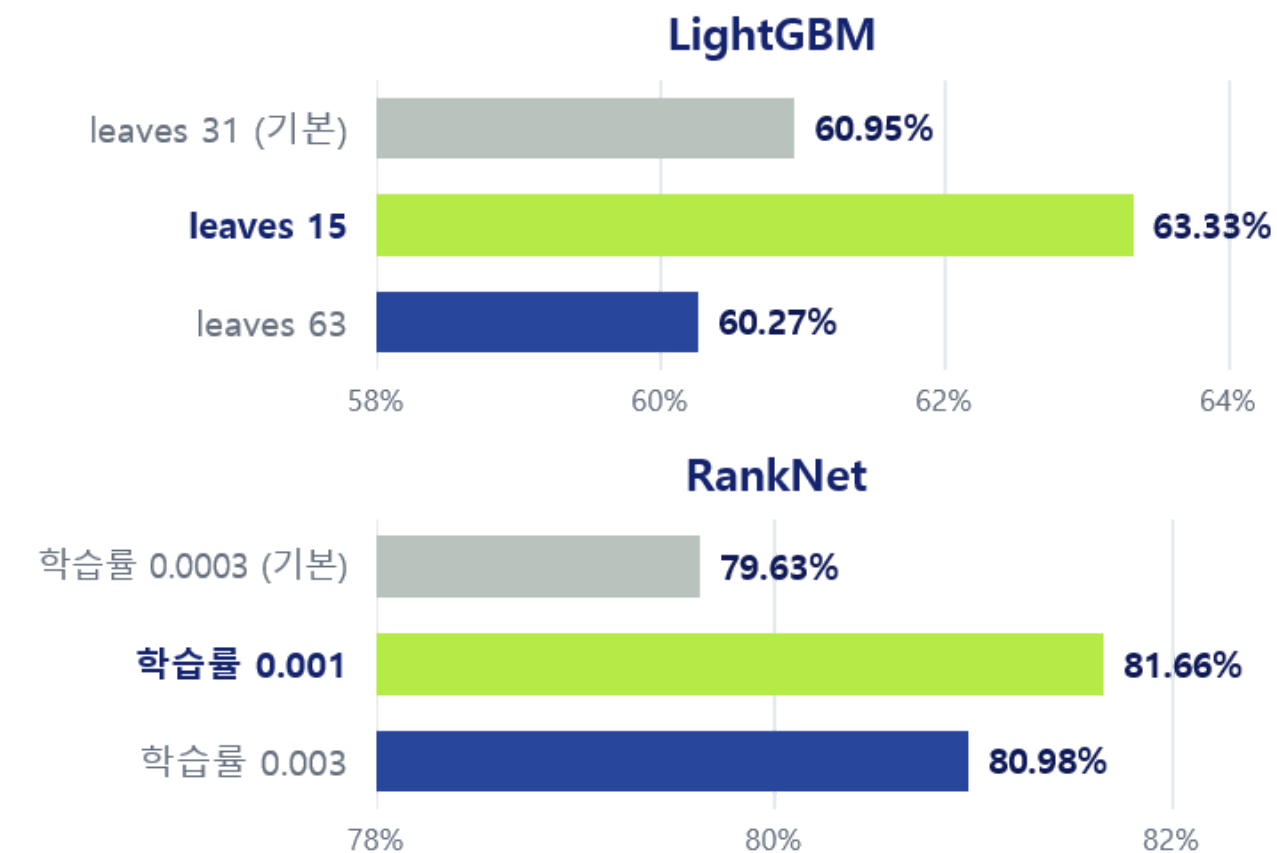
구분	요청 수	역할
학습	1,109	모델 학습
검증	589	검증 성능 및 학습 경향 확인
Warm-start 테스트	589	학습에 등장한 사용자의 이후 요청 평가
Cold-start 테스트	647	학습에 등장하지 않은 사용자 평가

추천 품질과 실행 비용 평가

지표	판단하는 내용	좋은 방향
Top-1 best utility	첫 추천이 utility가 가장 높은 후보인 요청 비율	높을수록 좋음
NDCG@3	상위 3개 추천의 relevance 순서와 품질	높을수록 좋음
Regret	최선 후보와 첫 추천 후보의 utility 차이	낮을수록 좋음
Pairwise accuracy	utility가 다른 후보 쌍의 순서를 맞힌 비율	높을수록 좋음

RankNet과 LightGBM 성능·자원 비교

설정별 Top-1 최적 경로 선택률



모델	변경 설정	Top-1 변화
LightGBM A	leaves 15 (기본 31)	+2.38%p
LightGBM B	leaves 63	-0.68%p
RankNet A	학습률 0.001(기준 0.0003)	+2.03%p
RankNet B	학습률 0.003	+1.35%p

평가 항목	RankNet 최고 추천 성능	LightGBM 운영 효율
Top-1 정확도	81.66%	63.33%
Pairwise Accuracy	93.56%	87.02%
학습 시간	12.2초	2.8초
추론 시간 / 요청	8.4ms	8.8ms
최대 메모리	약 508MB	약 285MB

성능 우선

RankNet으로 추천 성능의 상한 확인

운영 효율 우선

LightGBM으로 빠른 재학습과 낮은 메모리 사용

※ 해당 실험 설정 기준 : LightGBM은 단일스레드, RankNet은 Pytorch 기준 CPU 멀티스레드로 실행되어 자원 비교 시 설정 차이를 고려해야 합니다

모델 튜닝 및 재현성 검증

RankNet 튜닝

학습률·학습 횟수

lr 0.002·100 epoch

- lr 0.001~0.002 구간이 안정적, NDCG@3 기준 0.002에서 최고

은닉층 크기·드롭아웃

hidden 128·dropout 0.1

- dropout 0.0~0.2보다 dropout 0.1이 개선 효과

LightGBM

리프 수·트리 수

leaves 7·estimators 800

- Estimator 증가와 함께 성능 개선, leaves7·estimators 800에서 NDCG@3 0.9923 달성

최소 자식 샘플 수·규제

min_child 20·λ 1

- min_child 20·λ1에서 NDCG@3 0.9926으로 추가 튜닝 최고

4-seed 재현성 검증

RN10 주요 지표(4-seed 평균)

Top-1

84.00%

NDCG@3

0.997629

Pairwise

94.20%

RN10은 RN06 대비 Top-1·Regret·Pairwise

4/4 seed 우세, NDCG@3은 3/4 seed 우세

모델 선택 최종 테스트

모델	구분	NDCG@3 ↑	Top1 ↑	Regret ↓	Pairwise ↑
RN10	Validation	0.997914	84.38%	0.000845	94.19%
	Warm Test	0.995584	79.97%	0.001596	93.81%
	Cold Test	0.996206	82.84%	0.001701	93.64%
LG12	Validation	0.992571	68.25%	0.003956	88.74%
	Warm Test	0.989575	67.06%	0.005091	88.37%
	Cold Test	0.991097	67.23%	0.005328	88.41%

```

1 name: ranknet
2 params:
3   hidden_dim: 128
4   depth: 1
5   dropout: 0.1
6   epochs: 100
7   batch_size: 256
8   learning_rate: 0.002
9   weight_decay: 0.01
10  optimizer: adamw

```

```

1 model:
2   name: lightgbm_ranker
3   params:
4     n_estimators: 800
5     learning_rate: 0.05
6     num_leaves: 7
7     max_depth: -1
8     min_child_samples: 20
9     reg_lambda: 1.0
10    n_jobs: 1

```

RankNet 선택 근거

2차 튜닝에서 RN10이 핵심 4지표 모두 개선. 4개 seed 반복에서도 RN06 대비 우위를 대부분 유지

NDCG 0.99763 ± 0.00049

LightGBM 선택 근거

NDCG만 보면 LG12가 미세하게 높지만, LG11은 Top1/Regret/Painwise가 더 좋아 실제 최고 Utility 후보 선택에 유리

Top 1 69.44% · Regret 0.003652

05. 시연 & 트러블 슈팅

시연





트러블 슈팅 (1)



01

문제

모든 경로를 탐색한 후 조건을 확인하며
경로를 생성하는 방식에서
탐색 시간이 과하게 늘어나는 것을 확인

해결 방안

타원의 장축 길이가 같다는 점을 활용하고 NetworkX
그래프 탐색시에 비용을 측정하는 함수를 만들어
경로 생성의 시간을 단축함

02

문제

프론트, 서버, 알고리즘, AI 사이의 규격이
알고리즘과 AI 모델의 입출력 값에 따라 자주
변경되어야 하였음

해결 방안

서버/클라이언트간 통신 엔드포인트에서는
어댑터 패턴으로 구성해 변경에 유연하게 대응하였으
며, DB의 경우에는 JSONB 타입을 이용하였음

트러블 슈팅 (2)



문제

알고리즘으로 후보 경로를 만들 때,
시설 및 환경 정보가 제공되지 않으면
알고리즘이 추천을 하여도 사용자의 요구
가 반영되지 않은 경로가 만들어졌었음

해결 방안

경로를 생성하기 이전에 사용자가 요청한
시설과 환경에 대한 정보를 미리
그래프에 넣어놓고 경로 후보를 생성하는
방식으로 업데이트

한계와 개선방향



현재 한계

합성 데이터의 한계

→ 실제 사용자의 선택을 완전히 반영하기 어렵다

파일과 메모리의 데이터를 이용하여 경로를 생성

→ 메모리에 그래프 캐싱 데이터가 너무 과하게 들어가 서비스 범위가 넓어지면 탐색에 비용이 늘어나기 쉽다



개선 방향

실사용 데이터 수집·반영

→ 코스 클릭·선택·러닝 완료 등 실사용 행동 데이터를 추적

데이터베이스와 동적 그래프 생성

→ 사용자의 요청 또는 특정 범위의 요청에 맞는 그래프를 동적으로 생성하여 사용하고 캐싱해둘 수 있음

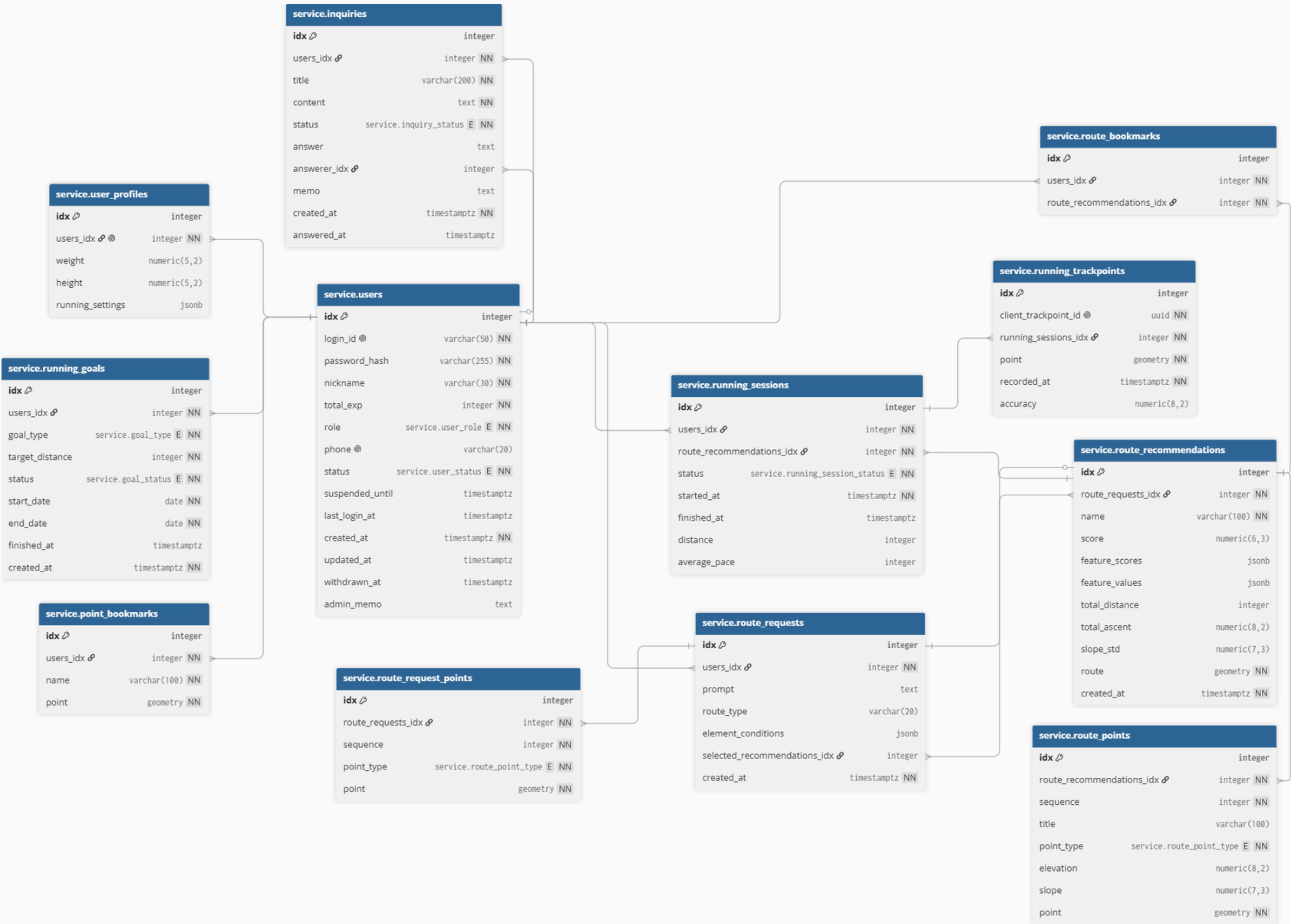
Q&A

Thank You

@dreamcanvakorea

Thank You

부록 1
ERD 전체



부록 2
1차 모델
실험 결과

모델	변경 파라미터	값	NDCG@3 ↑	Top1 ↑	Regret ↓	Pairwise ↑	Train	Infer/req	Peak RSS	Model size
Logistic	C	0.1	0.98041	55.52%	0.0101	83.59%	3.2s	10.3ms	327MB	8.4KB
Logistic	C	1	0.98027	55.35%	0.01012	83.65%	2.6s	9.0ms	336MB	8.4KB
Logistic	C	10	0.98027	55.35%	0.01012	83.64%	2.4s	8.6ms	329MB	8.4KB
Random Forest	min_samples_leaf	1	0.98515	63.50%	0.00511	86.26%	64.9s	34.1ms	563MB	277.8MB
Random Forest	min_samples_leaf	2	0.98662	63.50%	0.00552	86.43%	65.4s	22.1ms	401MB	120.9MB
Random Forest	min_samples_leaf	5	0.98671	62.99%	0.00572	85.94%	62.9s	30.5ms	333MB	44.5MB
LightGBM	num_leaves	15	0.98759	63.33%	0.00546	87.02%	2.8s	8.8ms	285MB	381.9KB
LightGBM	num_leaves	31	0.98767	60.95%	0.0062	86.45%	3.9s	9.5ms	289MB	727.7KB
LightGBM	num_leaves	63	0.98658	60.27%	0.00724	85.05%	3.8s	8.5ms	284MB	1.4MB
XGBoost	max_depth	3	0.9851	61.12%	0.00662	84.67%	5.8s	9.8ms	559MB	246.6KB
XGBoost	max_depth	6	0.9844	59.59%	0.00781	84.79%	6.7s	12.9ms	549MB	682.1KB
XGBoost	max_depth	9	0.9834	58.74%	0.00847	83.18%	9.5s	12.2ms	557MB	1.1MB
CatBoost	depth	4	0.982	60.10%	0.00763	85.05%	10.0s	10.9ms	302MB	135.1KB
CatBoost	depth	6	0.9829	59.42%	0.0073	84.65%	13.1s	12.0ms	301MB	288.8KB
CatBoost	depth	8	0.9804	57.56%	0.00828	83.58%	30.5s	11.9ms	311MB	892.7KB
RankNet	learning_rate	0.0003	0.99471	79.63%	0.00183	92.47%	12.4s	8.2ms	503MB	29.4KB
RankNet	learning_rate	0.001	0.99656	81.66%	0.00118	93.56%	12.2s	8.4ms	508MB	29.4KB
RankNet	learning_rate	0.003	0.99595	80.98%	0.00127	93.72%	13.7s	9.0ms	507MB	29.4KB

부록 3
3차 모델
복잡도 조절 결과

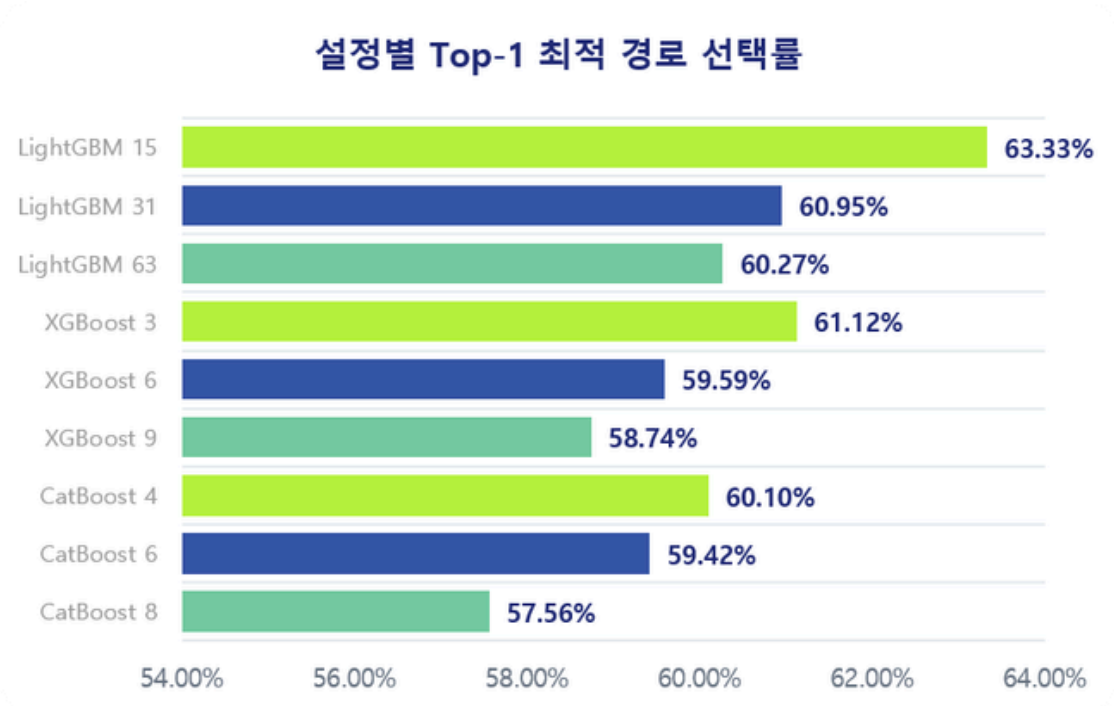
RankNet

설정	NDCG@3 ↑	Top1 ↑	Regret ↓	Pairwise ↑
hidden=64 / dropout=.1	0.9977	82.00%	0.001198	93.84%
hidden=32	0.99549	80.65%	0.001748	92.84%
hidden=128	0.99791	84.38%	0.000845	94.19%
dropout=0.0	0.99575	79.12%	0.001499	92.50%
dropout=0.2	0.99635	78.44%	0.001754	93.42%

LightGBM

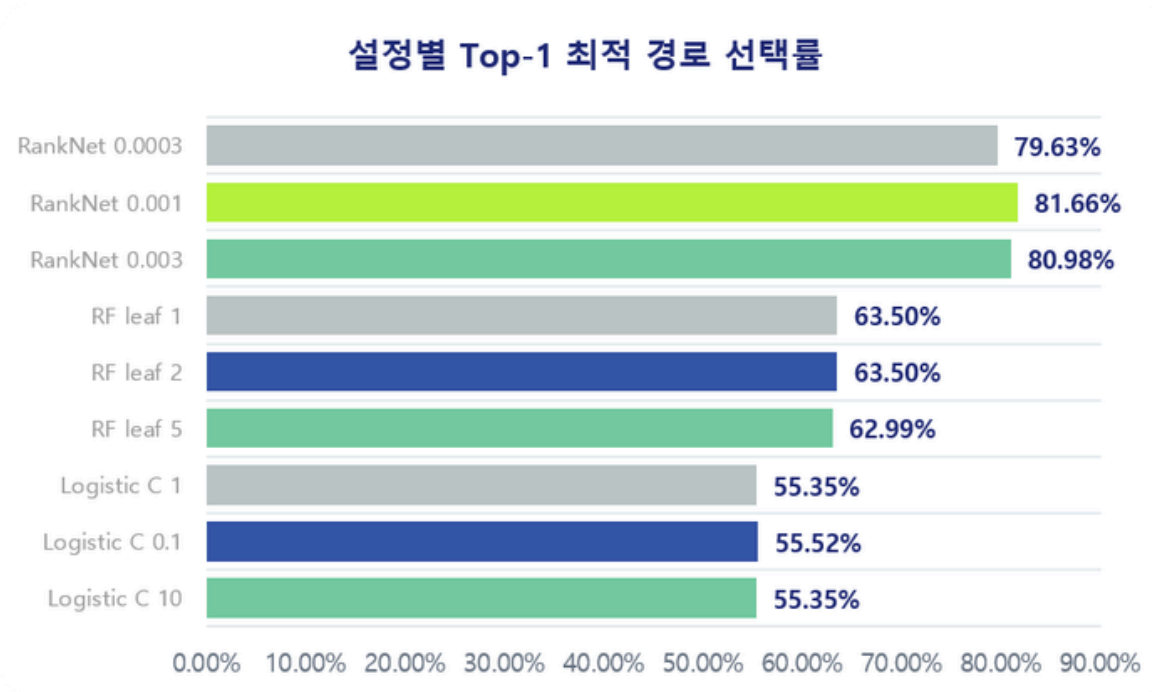
설정	NDCG@3 ↑	Top1 ↑	Regret ↓	Pairwise ↑
child=20 / $\lambda=0$	0.99234	66.21%	0.004003	88.91%
child=10	0.99243	67.74%	0.004186	89.03%
child=40	0.99169	67.06%	0.004156	88.81%
$\lambda=0.1$	0.99213	69.44%	0.003652	89.06%
$\lambda=1.0$	0.99257	68.25%	0.003956	88.74%

부록 4
모델 튜닝 결과



모델	변경 설정	Top-1 변화
LightGBM A	leaves 15 (기본 31)	+2.38%p
LightGBM B	leaves 63 (기본 31)	-0.68%p
XGBoost A	depth 3 (기본 6)	+1.53%p
XGBoost B	depth 9 (기본 6)	-0.85%p
CatBoost A	depth 4 (기본 6)	+0.68%p
CatBoost B	depth 8 (기본 6)	-1.86%p

LightGBM leaves 15: Top-1 63.33%, 학습 2.8초, 모델크기 381.9KB

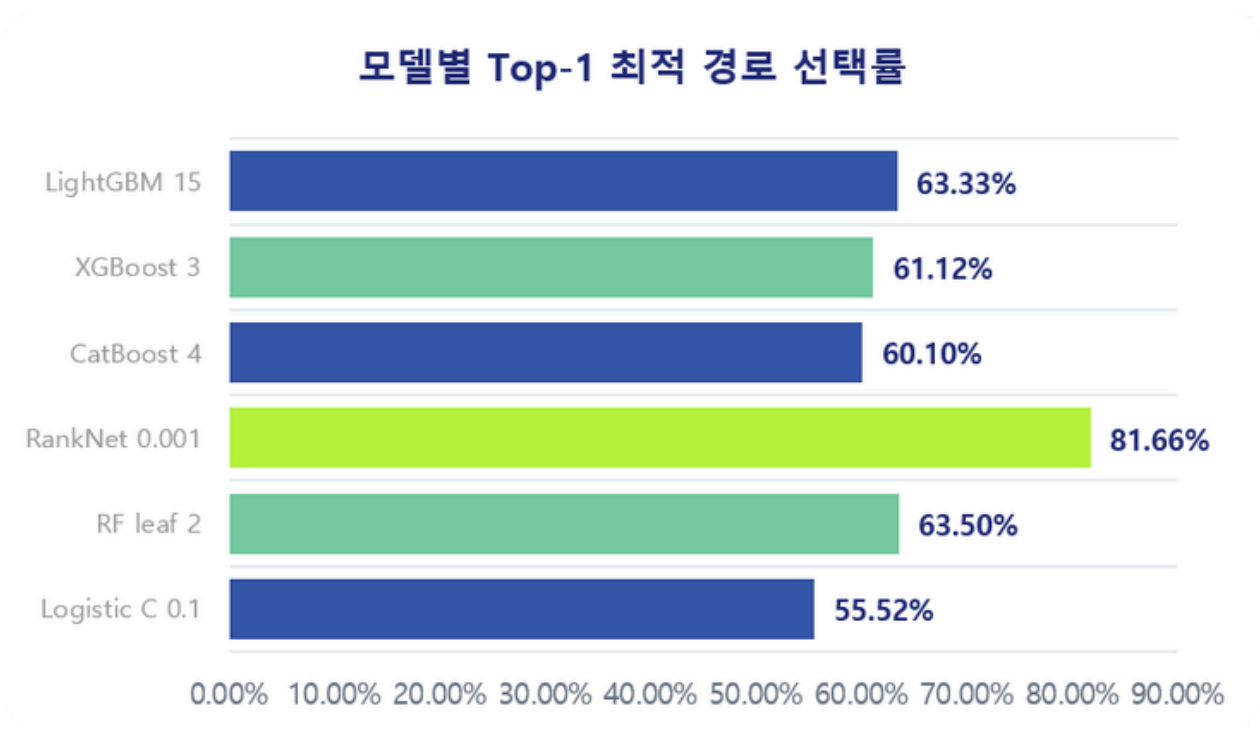


모델	변경 설정	Top-1 변화
RankNet A	학습률 0.001 (기준 0.0003)	+2.03%p
RankNet B	학습률 0.003 (기준 0.0003)	+1.35%p
RF A	leaf 2 (기준 1)	+0.00%p
RF B	leaf 5 (기준 1)	-0.51%p
Logistic A	C 0.1 (기본 1)	+0.17%p
Logistic B	C 10 (기본 1)	+0.00%p

RF leaf 2: Top-1 유지, 모델 277.8MB에서 120.9MB로 감소, 추론 34.1ms에서 22.1ms로 단축

부록 5
전체 성능 비교

✓ RankNet 학습률 0.001은 Top-1과 주요 품질 지표에서가장 우수한 결과를 보임



대표 설정	NDCG@3	Regret	Pairwise
LightGBM leaves 15	0.98759	0.00546	87.02%
XGBoost depth 3	0.9851	0.00662	84.67%
CatBoost depth 4	0.982	0.00763	85.05%
RankNet lr 0.001	0.99656	0.00118	93.56%
RF leaf 2	0.98662	0.00552	86.43%
Logistic C 0.1	0.98041	0.0101	83.59%

RankNetlr 0.001: 2위 RF leaf 2보다 Top-1 18.16%p 높고, Regret 0.00118로 가장낮음

부록 6

모델의 학습 강도 선택

RankNet

LR ↓ / Epoch →	60	100
0.0005	0.9959	0.996
0.001	0.996	0.9973
0.002	0.9967	0.9977
0.003	0.9964	0.9964

해석)
lr 0.001 ~ 0.002 영역이 안정적이었고,
주지표 기준 lr=0.002 / 100 epoch가 0.9977로 최고

1차 기본값 · RankNet
lr=0.002 · epochs=100

LightG BM

Estimators ↓ / Leaves →	7	15
100	0.9836	0.9871
300	0.9891	0.9879
500	0.991	0.9901
800	0.9923	0.9912

해석)
Estimator 증가와 함께 성능이 개선. 더 복잡한 leaves= 15보다
leaves=7 / estimators=800에서 0.9923으로 최고

1차 기본값 · LightG BM
leaves=7 · estimators=800

각각 8회씩 학습 강도에 영향을 주는 요소를 확인

부록 1

복잡도와 규제 강도 선택

RankNet · hidden_dim / dropout

Dropout → 은닉층 차원 ↓	0	0.1	0.2
32	-	0.9955	-
64	0.9958	0.9977	0.9964
128	-	0.9979	-

결론)

Dropout 변경을 통해 특별한 개선점을 찾지 못하였으며 은닉층의 표현력을 높이면 약간의 개선 효과가 있었음

LightGBM · min_child_samples / reg_lambda

L2 규제 → 리프 노드 최소 샘플	0	0.1	1
10	0.9924	-	-
20	0.9923	0.9921	0.9926
40	0.9917	-	-

결론)

dropout 0.0 또는 0.2는 개선을 만들지 못했다.

반면 hidden_dim =128은 NDCG, Top1, Regret, Pairwise를 모두 개선해 RN10을 다음 후보를 선택

부록 8

시드를 이용한 모델 안정성 확인

Seed별 NDCG@3

Candidate	7	21	42	84
RN06	0.997627	0.99582	0.997701	0.996088
RN10	0.996965	0.998067	0.997914	0.997568
LG11	0.992131	0.992131	0.992131	0.992131
LG12	0.992571	0.992571	0.992571	0.992571

RN 10 vs RN06

Top1 · Regret · Pairwise: 4/4 seeds 우세
NDCG@3: 3/4 seeds 우세

4-seed 평균 ± 표준편차

Candidate	NDCG@3	Top1	Regret ↓	Pairwise
RN06	0.996809 ± 0.000994	82.26% ± 0.45%p	0.001154 ± 0.000106	93.74% ± 0.14%p
RN10	0.997629 ± 0.000489	84.00% ± 0.94%p	0.000909 ± 0.000108	94.20% ± 0.14%p
LG11	0.992131 ± 0.000000	69.44% ± 0.00%p	0.003652 ± 0.000000	89.06% ± 0.00%p
LG12	0.992571 ± 0.000000	68.25% ± 0.00%p	0.003956 ± 0.000000	88.74% ± 0.00%p

재현성 판단

RN10은 평균 성능이 더 높고 NDCG 표준편차도 RN06보다 작다.
결정론적 모델인 LightGBM은 시드에 따른 차이가 없다